Discovery Engine

Product Specification for Project Management OS

AI-native ambiguity resolution from messy business problems to execution-ready projects

Prepared for	Simple.biz AI & Automation Team
Document type	Product / architecture specification
Date	June 26, 2026
Version	v0.1 working spec
Primary decision	Build Discovery Engine while external downstream credentials are blocked

One-line product thesis

The product is not an intake form with AI attached. It is an ambiguity-resolution engine that turns messy business problems into evaluated, approved, execution-ready projects.

Contents

1. Executive summary

The current intake is useful, but it is too strict for the deeper product vision. The goal should not be to make users complete a better form. The goal should be to let users describe ambiguity naturally, then have AI infer, clarify, structure, evaluate, and prepare the work for delivery.

Recommended path

Build the Discovery Engine now. It is a valid and high-leverage development path because it does not require live Monday or GitHub credentials, and it strengthens the front half of the system while preserving the evaluation, governance, and downstream architecture already built.

What changes

Old model	New model
User completes intake; AI checks it.	User explains a problem; AI creates the proposal.
The intake form is the front door.	The conversation is the front door.
AI triages and organizes the ask.	AI resolves ambiguity and proposes direction.
Evaluation starts from user-entered fields.	Evaluation starts from an AI-generated Project Proposal.
Downstream receives copied intake fields.	Downstream receives a deterministic Provisioning Manifest.

Target end-to-end flow

```
Messy user ask
  -> Discovery Engine
  -> Problem Frame
  -> Solution Options
  -> Selected Direction
  -> Project Proposal
  -> Evaluation Orchestrator
  -> Governance
  -> Provisioning Manifest
  -> Monday / GitHub / Credentials / Notifications
```

Core outcome

- The user can start with symptoms instead of requirements.
- The AI separates the underlying problem from the user-suggested solution.
- The system asks only decision-changing questions instead of forcing a long form.
- The existing evaluation orchestrator critiques a coherent proposal instead of raw conversation.
- The downstream layer stays deterministic and adapter-friendly.

2. Product reframe

The real job of intake

An intake exists because humans are messy and execution systems are precise. If AI can bridge that gap, the intake no longer needs to be a form. It becomes a guided discovery conversation that produces structure on behalf of the user.

Product identity

What the user says	What the AI should infer
“We need a chatbot.”	Possibly a support deflection, knowledge search, documentation, or ticket workflow problem.
“Invoices take forever.”	Possibly OCR, approval routing, accounting integration, or process cleanup.
“Client wants AI.”	Likely a discovery engagement before any implementation commitment.
“Need a dashboard.”	Possibly visibility, reporting, data quality, or KPI alignment.

What the user says	What the AI should infer
“Can we automate this?”	A business-process improvement opportunity that may or may not require software.

Design shift

The user does not need to know the project. The user only needs to describe the pain. The AI discovers whether there is a project, what type it is, whether software is the right response, and how it should be executed.

Two AI personalities

Discovery AI	Delivery AI
Conversational, consultant-like, ambiguity-tolerant.	Engineering-focused, deterministic, scope-aware.
Asks: What is happening? Who is affected? What outcome matters?	Asks: What architecture, tasks, dependencies, and risks exist?
Produces Problem Frame, Solution Options, Project Proposal.	Produces evaluation, governance decisions, execution plan, provisioning payloads.
Optimizes for understanding and direction.	Optimizes for feasibility, execution quality, and downstream correctness.

3. Current project state and why this path is valid

Current state map

Component	Status	Implication
Intake form / intake object	Exists or mostly exists	Keep it as a backend schema, legacy/manual mode, or admin correction layer.
Evaluation orchestrator	Exists	Move it one step later so it evaluates Project Proposals instead of raw intake.
Governance / approval logic	Exists or partially exists	Still valid after proposal generation.
Distribution plan	Exists conceptually	Can consume a deterministic Provisioning Manifest.
Downstream provisioning foundation	Partially built	Can be tested with mock manifests while live credentials are blocked.
Monday/GitHub live adapters	Externally blocked	Do not stall here; keep adapter contracts stable and continue front-half development.
Discovery Engine	New build path	Highest leverage path because it improves the product thesis and is not blocked externally.

Why this is valid development now

- **It unblocks product progress.** Discovery, proposal generation, evaluation handoff, and mock provisioning can all be built without external credentials.
- **It reduces integration risk later.** Adapters become consumers of a stable manifest instead of ad hoc outputs from a form or chat transcript.
- **It protects existing work.** The evaluation orchestrator, governance gates, and provisioning foundation remain useful.
- **It improves the product.** The system becomes AI-native rather than a stricter form with AI validation.

How the existing Dev Operations workspace remains the downstream model

The existing workspace structure remains valuable because it already separates work by size and purpose: people, projects, epics, sprint tasks, microtasks, and credentials. The Discovery Engine should not replace that model. It should generate the structured objects that map cleanly into it.

4. Discovery Engine specification

Core responsibility

The Discovery Engine sits before the existing evaluation orchestrator. Its responsibility is to transform raw, ambiguous human input into a structured Project Proposal that can be evaluated and eventually provisioned.

```
Conversation
  -> Intent Extraction
  -> Problem Framing
  -> Solution Generation
  -> Clarification Planning
  -> Direction Selection
  -> Proposal Composition
  -> Execution Mapping
```

Stage 1 - Raw conversation capture

- Capture the original user statement without forcing fields.
- Detect pain, urgency, user role, client/internal context, mentioned systems, and ambiguity.
- Preserve the raw conversation as evidence, but do not send raw conversation downstream.

```
{
  "raw_request": "Support keeps answering the same questions.",
  "detected_pain": "Repeated support workload",
  "mentioned_systems": [],
  "initial_confidence": 0.42
}
```

Stage 2 - Intent extraction

- Classify the ask as a software project, automation opportunity, dashboard/reporting need, AI assistant opportunity, process improvement, bug/fix, microtask, vague discovery request, duplicate, or not a project.
- Separate what the user requested from the underlying problem.
- Flag possible solution bias, such as assuming a chatbot is the answer before understanding the support problem.

Stage 3 - Problem framing

- Produce a clear problem statement.
- Identify affected users, current process, pain points, business impact, known constraints, assumptions, unknowns, and success criteria.
- Assign confidence by dimension rather than pretending full certainty.

```
{
  "problem_statement": "The support team repeatedly answers similar questions, creating avoidable
workload and slower response times.",
  "affected_users": ["Support team", "Requesters"],
  "success_criteria": ["Reduce repeated manual responses", "Improve answer consistency"],
  "unknowns": ["Audience", "Knowledge source", "Support channel"],
  "confidence": 0.72
}
```

Stage 4 - Solution generation

- Generate 2 to 4 plausible solution paths.
- Include when each option fits, when it is wrong, complexity, dependencies, risks, and expected upside.
- Recommend one option, but make alternatives visible for human judgment.

Option	When it fits	Complexity
Knowledge base cleanup	Answers already exist but are hard to find.	Low
AI knowledge assistant	Users need conversational access to approved content.	Medium
Ticket deflection workflow	Most questions arrive through helpdesk/ticket forms.	Medium/high

Option	When it fits	Complexity
Support analytics dashboard	Root cause is unclear and repeated themes need measurement.	Medium

Stage 5 - Clarification planner

- Ask only questions that materially change recommendation, scope, risk, or implementation plan.
- Classify unknowns as blocking, important, or deferred.
- Limit each turn to the smallest number of high-impact questions.

Preferred clarification style

Bad: “Who are the stakeholders? What is the budget? What is the timeline?”

Good: “Are these repeated questions coming from customers, internal staff, or both? That changes security, channel, and approval path.”

Stage 6 - Recommended direction

- Summarize the recommended solution.
- Explain why it fits the problem.
- Show alternatives considered.
- Give the user simple choices: approve direction, edit direction, compare alternatives, send to backlog, or mark as not a project.

Stage 7 - Proposal composer

- Create the structured Project Proposal object.
- Include problem, recommended solution, objectives, scope, out-of-scope, stakeholders, systems, risks, success metrics, assumptions, unknowns, and suggested decomposition.
- Make the proposal editable before evaluation.

Stage 8 - Execution mapper

- Translate the proposal into a provisioning-ready plan.
- Do not directly create Monday rows or GitHub repos from Discovery.
- Generate a deterministic manifest that the downstream provisioning layer can consume later.

5. Internal agent model

The Discovery Engine should not be one giant prompt. Use smaller modules with clear responsibilities and structured outputs.

Agent / Module	Purpose	Primary output
Conversation Agent	Keeps the user talking naturally; summarizes what was heard; avoids form behavior.	Conversation summary, ambiguities, useful quotes
Intent Extraction Agent	Determines what kind of ask this is and whether it should become a project, task, microtask, duplicate, or non-project.	Intent type and routing recommendation
Problem Framing Agent	Converts symptoms into a business problem and separates problem from requested solution.	ProblemFrame
Existing Context Agent	Checks whether this matches existing projects, epics, tasks, credentials, or microtasks.	Duplicate or linkage candidates
Solution Generation Agent	Creates multiple plausible solution paths and tradeoffs.	SolutionOption list
Clarification Agent	Asks only high-impact questions and ranks unknowns.	Question plan and unknown classification
Proposal Composer Agent	Builds the evaluation-ready proposal.	ProjectProposal
Execution Mapper Agent	Maps the proposal to Projects, Epics, Tasks, Credentials, GitHub, and notifications.	ProvisioningManifest draft

Routing outcomes

- Create new project.
- Create epic under existing project.
- Create sprint task under existing epic/project.
- Create microtask.
- Request more discovery.
- Mark as duplicate.
- Recommend process-only change.
- Reject, defer, or archive.

6. Discovery state machine and confidence model

State machine

```
draft
  -> conversation_started
  -> intent_detected
  -> problem_framed
  -> solutions_generated
  -> clarification_needed
  -> direction_selected
  -> proposal_generated
  -> evaluation_ready
  -> sent_to_evaluation
```

State	Meaning
draft	User has opened a discovery but has not provided enough input.
conversation_started	Raw ask exists.
intent_detected	System has classified likely ask type.
problem_framed	System has a working problem statement.
solutions_generated	System has produced possible solution paths.
clarification_needed	System needs targeted answers or user confirmation.
direction_selected	User/admin has selected or confirmed a solution direction.
proposal_generated	Structured proposal exists.
evaluation_ready	Proposal meets minimum completeness threshold.
sent_to_evaluation	Existing orchestrator takes over.

Side states

- duplicate_candidate
- not_a_project
- microtask_candidate
- needs_human_review
- parked
- abandoned

Confidence dimensions

```
{
  "confidence": {
    "problem_understanding": 0.82,
    "solution_fit": 0.74,
    "scope_clarity": 0.61,
    "technical_feasibility": 0.68,
    "stakeholder_clarity": 0.55,
    "downstream_mapping": 0.71
  }
}
```

Confidence range	Behavior
0.00-0.40	Keep discovering; do not generate a proposal.
0.41-0.65	Generate a rough frame and ask clarifying questions.
0.66-0.80	Generate a proposal with visible assumptions.
0.81-1.00	Generate proposal and recommend evaluation.

Core object model

Object	Purpose
DiscoverySession	Stateful conversation record, current summary, confidence, unknowns, and timeline events.
ProblemFrame	The understood business problem, current process, impact, success criteria, assumptions, and unknowns.
SolutionOption	One possible way to solve the problem, including pros, cons, dependencies, risks, and rank.
ProjectProposal	The main evaluation handoff object; replaces user-filled intake as the first-class artifact.
ProvisioningManifest	The deterministic downstream contract for Monday, GitHub, credentials, and notifications.

```
ProjectProposal {
  id
  discovery_session_id
  selected_solution_id
  title
  problem_statement
  recommended_solution
  objectives[]
  scope: { in_scope[], out_of_scope[] }
  stakeholders[]
  systems[]
  risks[]
  success_metrics[]
  assumptions[]
  unknowns[]
  suggested_epics[]
  suggested_tasks[]
  status
}
```

7. UI considerations

Main UI concept

Start with one field: “What are you trying to accomplish?”

Do not start with project name, budget, stakeholders, requirements, and timeline. The first interaction should feel like talking to a consultant, not submitting a ticket.

Recommended three-panel workspace

```
+----------------------+------------------------------+----------------------+
| Discovery Timeline   | Conversation                 | AI Understanding     |
|                      |                              |                      |
| - Raw ask            | User + AI chat               | Problem              |
| - Intent detected    |                              | Goals                |
| - Problem framed     |                              | Unknowns             |
| - Solutions          |                              | Confidence           |
| - Proposal           |                              | Suggested next step  |
+----------------------+------------------------------+----------------------+
```

Left panel - Discovery Timeline

- Shows progress through discovery states.
- Makes the process feel structured without exposing a form.
- Example: Raw ask captured -> Intent detected -> Problem framed -> Solutions generated -> Direction selected -> Proposal ready -> Sent to evaluation.

Center panel - Conversation

- Primary user surface.
- Supports natural responses like “yea option 2 first” or “close but this is for clients not internal staff.”
- AI should summarize, hypothesize, and ask focused questions.

Right panel - AI Understanding

- Live-updating structured summary.
- Shows problem, likely solution, assumptions, unknowns, confidence, and next step.
- Makes AI behavior auditable without exposing private reasoning.

Key UI patterns

Pattern	Purpose
AI hypothesis card	Shows “Here is what I think you mean” after the first useful user message.
Solution comparison cards	Presents 2-4 options with effort, risk, upside, dependencies, and recommendation.
Editable proposal	Lets users accept, edit, regenerate, or ask why for each proposal section.
Minimum questions mode	Signals “I only need two answers before this is evaluation-ready.”
Send to Evaluation handoff	Clearly transitions from Discovery AI to Delivery AI/evaluation agents.
Not a project path	Allows the AI to recommend no build, a microtask, a process change, or backlog parking.

First UI priority

Make one moment feel magical: the user types a vague business problem, and the system understands it better than a form would have.

8. Downstream mapping and handoff contract

Discovery output to Dev Operations workspace

The downstream workspace remains the execution structure. Discovery should generate the correct work objects for that structure rather than dumping an intake into a single board.

Discovery output	Dev Ops destination
Recommended project	Projects Portfolio
Major solution areas	Roadmap & Epics
Concrete implementation work	Sprint Tasks
Small standalone follow-ups	Microtasks & Ops
API keys, logins, service accounts, access needs	Credentials Vault
Suggested owner/team fit	Team Directory reference
Technical dependencies	Project fields plus Credential entries
Unclear but important assumptions	Proposal unknowns / review notes
Not worth building	No project; optionally microtask, process recommendation, or archive

Handoff contract

The Discovery Engine should not directly create external objects. It should produce a ProvisioningManifest that downstream adapters consume. This keeps the front half testable while Monday/GitHub credentials are unavailable.

```
{
  "manifest_version": "1.0",
  "source": "discovery_engine",
  "proposal_id": "prop_001",
  "recommended_action": "create_project",
  "monday": {
    "projects_portfolio": { "create": true, "name": "Internal Support Knowledge Assistant" },
    "roadmap_epics": ["Knowledge Source Integration", "Assistant Experience", "Governance and Safety"],
    "sprint_tasks": ["Confirm approved knowledge sources", "Design retrieval architecture", "Build
prototype assistant flow"],
    "credentials_vault": ["Knowledge source API/service account"],
    "microtasks_ops": []
  },
  "github": {
    "create_repo": true,
    "repo_name": "internal-support-knowledge-assistant",
    "labels": ["discovery", "backend", "frontend", "integration", "security"]
  },
  "notifications": { "send_approval_summary": true }
}
```

Example generated execution map

Object type	Example
Project	Internal Support Knowledge Assistant
Epics	Knowledge Source Integration; Assistant Experience; Governance and Safety
Sprint tasks	Identify approved knowledge sources; Design retrieval flow; Build prototype assistant; Add answer review controls
Credentials	Knowledge source API/service account; support channel integration access
GitHub	Repo, labels, and optionally seed issues generated from accepted tasks

9. MVP scope and development phases

MVP goal

A user can describe a messy problem, and the system produces an evaluation-ready Project Proposal.

MVP flow

```
User starts discovery
  -> AI extracts intent
  -> AI frames problem
  -> AI proposes 2-3 solution options
  -> User selects direction
  -> AI generates Project Proposal
  -> Existing evaluator reviews proposal
  -> System generates mock ProvisioningManifest
```

MVP out of scope

- Live Monday creation.
- Live GitHub creation.
- Real credential provisioning.
- Perfect sprint planning.
- Full team capacity logic.
- Complex multi-user collaboration.

- Voice input.
- Deep historical memory.

Development phases

Phase	Build	Success criteria
1. Discovery core	DiscoverySession, conversation endpoint, intent extraction, problem framing, confidence, unknowns.	Given a vague ask, the system summarizes the problem and asks useful follow-ups.
2. Solution options	Generate solution options, comparison cards, recommendation, selection/editing.	Given a framed problem, system proposes plausible approaches and recommends one.
3. Proposal composer	ProjectProposal schema, generation, editable proposal UI, validation.	Selected direction becomes a complete structured proposal.
4. Evaluation integration	Adapter to existing evaluation orchestrator, result display, revise/resubmit loop.	Evaluation agents review AI-generated proposals without raw intake fields.
5. Mock downstream	ProvisioningManifest generator, Monday/GitHub/credentials mock payloads, run log.	Approved proposal produces deterministic downstream payloads without live credentials.
6. Live adapters	Monday project/epic/task creation, GitHub repos/issues/labels, notification delivery.	Approved proposal becomes real execution workspace after credentials are available.

10. Implementation backlog

Epic 1 - Discovery Session Foundation

- Create DiscoverySession model.
- Add discovery status enum.
- Store raw user messages and AI summaries.
- Add confidence object and unknowns array.
- Add session timeline events.

Epic 2 - Intent and Problem Framing

- Build intent extraction prompt/module.
- Build problem framing prompt/module.
- Add ProblemFrame schema.
- Extract assumptions and unknowns.
- Add confidence scoring.
- Add not-a-project, microtask, existing-project, and duplicate classification.

Epic 3 - Clarification Engine

- Generate decision-changing questions.
- Rank questions by impact.
- Limit questions per turn.
- Track answered questions.
- Recompute confidence after answers.
- Support skip, assume, or decide later.

Epic 4 - Solution Options

- Generate 2-4 solution options.
- Add pros, cons, risks, and dependencies.
- Rank recommended option.
- Add solution comparison UI.
- Allow user to select or edit direction.

Epic 5 - Proposal Composer

- Create ProjectProposal model.

- Generate proposal from selected solution.
- Add editable proposal UI.
- Add proposal completeness validator.
- Add evaluation-ready state.

Epic 6 - Evaluation Handoff

- Map ProjectProposal to current evaluation input.
- Trigger existing evaluation orchestrator.
- Store evaluation result against proposal.
- Add revise/resubmit loop.
- Display evaluation summary in UI.

Epic 7 - Mock Provisioning Manifest

- Generate Monday payload.
- Generate GitHub payload.
- Generate credentials payload.
- Generate microtasks payload.
- Add provisioning preview UI.
- Add run log.
- Add ready-for-live-adapter status.

11. Product rules and naming recommendations

Product rules

Rule	Meaning
The user gives symptoms; AI proposes structure.	Do not punish vague input. Vague input is the point.
Ask only decision-changing questions.	Every clarification should have a reason connected to scope, risk, recommendation, or implementation.
Separate problem from proposed solution.	If the user asks for a chatbot, validate whether the real problem is support deflection, knowledge access, or something else.
Make assumptions visible.	The AI can infer, but it should not silently pretend.
Downstream consumes structured contracts.	Monday and GitHub should receive approved manifests, not raw conversation.
Not every discovery becomes a project.	Possible outcomes include project, epic, task, microtask, duplicate, process change, defer, or reject.

Naming recommendations

Current / form-heavy language	Recommended language
Submit Intake	Start Discovery
Intake	Discovery / Opportunity / Project Proposal
Requirements	AI Understanding / Working Assumptions
Approval	Approve for Evaluation / Approve for Delivery
Distribution Plan	Execution Plan / Provisioning Manifest
Submit to Monday	Provision Workspace

Recommended button flow

```
Start Discovery
  -> Choose Direction
  -> Generate Proposal
  -> Send to Evaluation
  -> Approve for Delivery
  -> Provision Workspace
```

12. References and source context

- Source context used: Dev Operations Workspace Manager's Guide, Simple.biz AI & Automation Team, prepared by Cob. Key downstream concepts used here include Team Directory, Projects Portfolio, Roadmap & Epics, Sprint Tasks, Credentials Vault, and Microtasks & Ops.
- Current design assumption: Projects Portfolio remains the hub, Epics remain the planning layer, Sprint Tasks remain the execution layer, Microtasks remain small standalone work, and Credentials Vault remains the access register.
- This specification is a working product/architecture draft intended to guide implementation while external Monday/GitHub access is unavailable.

Final recommendation

Proceed with Discovery Engine development now. Treat the existing intake as a backend schema/manual mode, move evaluation behind Project Proposal generation, and use mock ProvisioningManifests until live downstream credentials are available.